

# MGMT 803, MBA@RICE, 2026

Deploying agents

Kerry Back

J. Howard Creekmore Professor of Finance

# This session

- 17. Academic Studio
- 18. Running apps and agents locally
- 19. Git and GitHub
- 20. Deploying apps and agents

# 17 • ACADEMIC STUDIO

VS Code with Claude Code built in, at [academic-studio.com](https://academic-studio.com).

# What it is

Under the hood it is VSCodium, the open-source build of VS Code, simplified for getting work done.

Claude Code opens on startup. A Claude menu sits in the menu bar, the other menus and toolbars are trimmed, and the Help menu is rebuilt around getting started.

A paid Anthropic account — Pro, Max, or API — is required to use Claude Code in it. Type `/login` in the prompt window the first time.

# Why are we installing it?

- It is an easy way (I hope) to install Python and key Python libraries.
- I expect you to get Claude accounts paid by Rice soon.
- You do not need a paid Claude account for this course.
- But, if you do have one, you can use Academic Studio instead of lab803.

# What comes with it

## Documents

The `xlsx`, `docx`, `pptx` and `pdf` skills, bundled and configured. LaTeX and Quarto for typeset output.

## A browser

The Playwright connector, set up for you. Claude can open pages, click through them, fill forms, and take screenshots.

## A toolchain

Python and the scientific stack, Jupyter, R, Quarto, TinyTeX, Node.js.

Help → Run Setup asks which profile you are — faculty, or students and professionals — and installs the supporting programs in one click.

# What carries over

- 1 Open a folder, and your files and your conversation history stay with that project.
- 2 Skills, connectors and `CLAUDE.md` files are shared with the Claude Code CLI and with the Code mode of Claude Desktop.
- 3 Conversation history is shared with the CLI.

Every Claude skill, connector and plugin works here.

macOS

Apple Silicon, M1 or later — it does not run on Intel Macs

Windows

x64, most machines

Windows ARM

Surface Pro and similar

Download from [academic-studio.com](https://academic-studio.com). Every release is also on the GitHub releases page.

## Getting it

# 18 · RUNNING APPS AND AGENTS LOCALLY

# Download from lab803

- 1 Ask Claude what to download. It knows which files the app needs and which are scratch.
- 2 Download them from the Files tab, into a folder on your own machine.
- 3 Open that folder in Academic Studio.

Your keys are not in those files, and should not be. Set `ANTHROPIC_API_KEY` and `OPENAI_API_KEY` on your own machine, the same way you set them in the container.

# What is already installed, and what is not

Run Setup gives you Python, `fastapi` and `uvicorn`. Four libraries are not in it:

```
1 python -m pip install anthropic openai claude-agent-sdk python-dotenv
```

`anthropic` for the chatbot on your Anthropic key, `openai` for the OpenRouter version, `claude-agent-sdk` for the agent, and `python-dotenv` for the line that reads your keys out of `env.txt`. On macOS the command is `python3 -m pip ...`. You can also just ask Claude to install them.

# Run it

## WINDOWS

```
1 python -m uvicorn app:app --port 8000
```

## MAC

```
1 python3 -m uvicorn app:app --port 8000
```

Then open `http://127.0.0.1:8000` in a browser.

`app:app` is the file and the FastAPI object inside it — `explorer:app` for `explorer.py`.

# 19 • GIT AND GITHUB

Keeping every version of the work, and putting the four files on the web.

# What a commit is

A version control system tracks every change you make. A commit is a snapshot you can return to.

## Install

Git is on lab803 already, and on your laptop with Academic Studio. Claude Code installs it anywhere it is missing.

## Commits

Ask it to commit, and the change is saved with a description of what it was.

## History

Ask when something changed, what changed it, or what the file looked like three versions ago.

# Try it: commit and restore

- 1 In lab803, tell Claude Code to create a new folder.
- 2 Upload a file into that folder from the Files tab.
- 3 Tell Claude to initialize the folder as a Git repository and commit the file.
- 4 Delete the file.
- 5 Ask Claude to restore it.

Commit changes regularly, so you can easily undo and roll back.

# GitHub

GitHub hosts Git repositories. The project lives in the cloud, backed up and shareable.

## PUSHING

With a free account, Claude Code pushes your repository to GitHub and pushes later commits when you ask. It uses the GitHub CLI.

## COLLABORATION

Others can clone it, propose changes, and open issues. Working alone, it is backup and deployment.

# Try it: push to GitHub

- 1 Create a free account at `github.com`.
- 2 Settings → Developer settings → Personal access tokens → Tokens (classic). Tick the box marked `repo`, generate, and copy it — GitHub shows it once.
- 3 Add a line `GH_TOKEN=` and the token to `env.txt`, and upload it to your container again from the Files tab.
- 4 Tell Claude the token is in `env.txt`, and to create a new GitHub repository and push the folder you committed earlier.
- 5 Open `github.com/your-user-name` and look at what arrived: the file, the commit, and the message Claude wrote.

# Before you push

If you make your repository public, then anyone can read it.

## Keep out

API keys, passwords, anything under a data agreement, and anything with a name in it. Tell Claude to list those files in a `.gitignore`, and Git will leave them out of every commit.

## Remember

Deleting a file in a later commit does not remove it. It stays in the history.

# GitHub Pages

Any repository becomes a website. No server, no configuration, no cost (but it is available for free only if your repo is public).

- 1 Tell Claude to create a page for you (with whatever you want on it) named `index.html`.
- 2 Commit and push again. Push the repository to GitHub.
- 3 Settings → Pages, and set the source to the `main` branch, either its root or a `docs/` folder.
- 4 The site is live at `yourusername.github.io/repo-name/`.

## OR ASK CLAUDE TO TURN IT ON

```
1 gh api -X POST repos/{owner}/{repo}/pages -F 'source[branch]=main' -F 'source[path]=/'
```

A custom domain costs about \$15 a year and replaces the `github.io` address.

# 20 · DEPLOYING APPS AND AGENTS

# Platform as a service

You hand a host your repository. It builds the app, runs it, and puts it behind a public URL. There is no server for you to administer.

Railway, Render, Koyeb, Fly and Heroku all do this. We use Koyeb, which deploys straight from a GitHub repository and has a free instance.

The free instance is one web service, 512 MB of memory, in Frankfurt or Washington. Koyeb asks for a credit card when you sign up, to stop abuse, and does not charge for it.

# Create an account and get a token

- 1 Create an account at `koyeb.com`.
- 2 In the control panel, create an API access token. Koyeb shows it once, so copy it then.
- 3 Add a line `KOYEB_TOKEN=` and the token to `env.txt`, and upload it again from the Files tab.
- 4 Tell Claude your Koyeb token is in `env.txt`.

The Koyeb CLI is already installed in your container. From here on Claude can do the rest — creating the service, reading the logs, and giving you the URL.

# Breakout exercise

- 1 Pick an app, chatbot, or agent that you have created. Do not use one with a paid API key (so no Anthropic key).
- 2 Tell Claude you want to deploy the app on Koyeb and your Koyeb access token is in env.txt.
- 3 If Claude asks you to do anything, reply “Can you do that instead?” If needed, tell Claude that the Koyeb CLI is installed.
- 4 When Claude says it is deployed, ask for the URL.
- 5 Open the URL on your laptop or phone.